

Java Collections Framework

1. ArrayList

```
ArrayList<Integer> list = new ArrayList<>();

list.add(x);           // Add at end

list.add(i, x);        // Add at index

list.get(i);           // Get element

list.set(i, x);        // Update element

list.remove(i);        // Remove by index

list.remove(Integer.valueOf(x)); // Remove by value

list.contains(x);      // Check element

list.indexOf(x);       // First index

list.lastIndexOf(x);   // Last index

list.size();           // Number of elements

list.isEmpty();        // Check empty

list.clear();          // Remove all
```

2. LinkedList

```
LinkedList<Integer> list = new LinkedList<>();

list.add(x);           // Add at end

list.add(i, x);        // Add at index

list.addFirst(x);      // Add at beginning

list.addLast(x);       // Add at end

list.get(i);           // Get by index

list.getFirst();       // Get first

list.getLast();        // Get last

list.set(i, x);        // Update

list.remove(i);        // Remove by index
```

```
list.remove(Integer.valueOf(x)); // Remove by value

list.removeFirst();           // Remove first

list.removeLast();           // Remove last

list.contains(x);             // Check element

list.indexOf(x);              // First index

list.lastIndexOf(x);         // Last index

list.peek();                  // Get first without removing

list.peekFirst();            // Get first without removing

list.peekLast();             // Get last without removing

list.poll();                  // Remove and return first

list.pollFirst();            // Remove and return first

list.pollLast();             // Remove and return last

list.offer(x);                // Add at end

list.offerFirst(x);          // Add at beginning

list.offerLast(x);           // Add at end

list.size();                  // Number of elements

list.isEmpty();              // Check empty

list.clear();                 // Remove all
```

3. HashSet

```
HashSet<Integer> set = new HashSet<>();

set.add(x);                   // Add element

set.remove(x);                // Remove element

set.contains(x);              // Check element

set.size();                   // Number of elements

set.isEmpty();                // Check empty

set.clear();                  // Remove all

set.iterator();               // Get iterator
```

Important: No duplicates, no guaranteed order.

4. LinkedHashSet

```
LinkedHashSet<Integer> set = new LinkedHashSet<>();
```

```
set.add(x);           // Add element
set.remove(x);        // Remove element
set.contains(x);       // Check element
set.size();           // Number of elements
set.isEmpty();         // Check empty
set.clear();           // Remove all
set.iterator();        // Get iterator
```

Important: No duplicates + maintains insertion order.

5. TreeSet

```
TreeSet<Integer> set = new TreeSet<>();
```

```
set.add(x);           // Add element
set.remove(x);        // Remove element
set.contains(x);       // Check element
set.first();          // Smallest element
set.last();           // Largest element
set.lower(x);         // Largest element < x
set.higher(x);        // Smallest element > x
set.floor(x);         // Largest element <= x
set.ceiling(x);       // Smallest element >= x
set.pollFirst();      // Remove + return smallest
set.pollLast();       // Remove + return largest
set.descendingSet();  // Reverse-order view
set.size();           // Number of elements
set.isEmpty();         // Check empty
set.clear();           // Remove all
```

Important: No duplicates + automatically sorted.

MAPS

6. HashMap

```
HashMap<Integer, String> map = new HashMap<>();

map.put(key, value);           // Add / update

map.putIfAbsent(key, value);   // Add if key doesn't exist

map.get(key);                  // Get value

map.getDefault(key, defaultValue); // Get or default

map.containsKey(key);          // Check key

map.containsValue(value);       // Check value

map.remove(key);                // Remove by key

map.remove(key, value);         // Remove if both match

map.keySet();                  // Get all keys

map.values();                   // Get all values

map.entrySet();                // Get key-value pairs

map.size();                     // Number of pairs

map.isEmpty();                 // Check empty

map.clear();                    // Remove all
```

Important: Stores key → value, keys are unique.

7. LinkedHashMap

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

map.put(key, value);           // Add / update

map.putIfAbsent(key, value);   // Add if key absent

map.get(key);                  // Get value

map.getDefault(key, defaultValue); // Get or default

map.containsKey(key);          // Check key

map.containsValue(value);       // Check value

map.remove(key);                // Remove key
```

```
map.remove(key, value);          // Remove if both match

map.keySet();                    // All keys

map.values();                    // All values

map.entrySet();                  // All entries

map.size();                      // Number of pairs

map.isEmpty();                   // Check empty

map.clear();                     // Remove all
```

Important: Same basic operations as HashMap + maintains insertion order.

8. TreeMap

```
TreeMap<Integer, String> map = new TreeMap<>();

map.put(key, value);             // Add / update

map.putIfAbsent(key, value);     // Add if key absent

map.get(key);                    // Get value

map.getDefault(key, defaultValue); // Get or default

map.containsKey(key);            // Check key

map.containsValue(value);        // Check value

map.remove(key);                 // Remove key

map.firstKey();                  // Smallest key

map.lastKey();                   // Largest key

map.lowerKey(key);               // Largest key < key

map.higherKey(key);              // Smallest key > key

map.floorKey(key);               // Largest key <= key

map.ceilingKey(key);             // Smallest key >= key

map.firstEntry();                // Entry with smallest key

map.lastEntry();                 // Entry with largest key

map.lowerEntry(key);             // Entry with key < key

map.higherEntry(key);            // Entry with key > key

map.floorEntry(key);             // Entry with key <= key
```

```
map.ceilingEntry(key);           // Entry with key >= key
map.pollFirstEntry();           // Remove smallest entry
map.pollLastEntry();            // Remove largest entry
map.keySet();                   // All keys
map.values();                   // All values
map.entrySet();                 // All entries
map.descendingMap();            // Reverse-order map
map.size();                     // Number of pairs
map.isEmpty();                 // Check empty
map.clear();                   // Remove all
```

Important: Key-value pairs + keys automatically sorted.

QUEUE

9. Queue

```
Queue<Integer> queue = new LinkedList<>();
```

```
queue.offer(x);                // Add element
queue.add(x);                  // Add element
queue.peek();                  // Get first without removing
queue.element();               // Get first without removing
queue.poll();                  // Remove + return first
queue.remove();                // Remove + return first
queue.contains(x);             // Check element
queue.size();                  // Number of elements
queue.isEmpty();               // Check empty
queue.clear();                 // Remove all
```

For DSA most imp:

```
queue.offer(x);
queue.peek();
```

```
queue.poll();
```

Queue = FIFO

First In → First Out

DEQUE

10. ArrayDeque

```
Deque<Integer> deque = new ArrayDeque<>();
```

```
deque.addFirst(x);    // Add at beginning
```

```
deque.addLast(x);     // Add at end
```

```
deque.offerFirst(x);  // Add at beginning
```

```
deque.offerLast(x);   // Add at end
```

```
deque.peekFirst();    // Get first
```

```
deque.peekLast();     // Get last
```

```
deque.pollFirst();    // Remove + return first
```

```
deque.pollLast();     // Remove + return last
```

```
deque.removeFirst();  // Remove first
```

```
deque.removeLast();   // Remove last
```

```
deque.getFirst();     // Get first
```

```
deque.getLast();      // Get last
```

```
deque.contains(x);    // Check element
```

```
deque.size();         // Number of elements
```

```
deque.isEmpty();      // Check empty
```

```
deque.clear();        // Remove all
```

Deque = Double Ended Queue

← Add / Remove Add / Remove →

[10 20 30]

PRIORITY QUEUE

11. PriorityQueue

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.add(x);           // Add element
```

```
pq.offer(x);         // Add element
```

```
pq.peek();           // Smallest element
```

```
pq.poll();           // Remove + return smallest
```

```
pq.remove(x);         // Remove specific element
```

```
pq.contains(x);       // Check element
```

```
pq.size();            // Number of elements
```

```
pq.isEmpty();         // Check empty
```

```
pq.clear();           // Remove all
```

Min Heap

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.offer(30);
```

```
pq.offer(10);
```

```
pq.offer(20);
```

```
pq.peek();           // 10
```

Max Heap

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
```

```
pq.offer(30);
```

```
pq.offer(10);
```

```
pq.offer(20);
```

```
pq.peek();           // 30
```

STACK

12. Stack

```
Stack<Integer> stack = new Stack<>();
```

```
stack.push(x);          // Add to top
stack.pop();            // Remove + return top
stack.peek();           // Get top
stack.empty();          // Check empty
stack.isEmpty();        // Check empty
stack.search(x);        // Search element
stack.contains(x);      // Check element
stack.size();           // Number of elements
stack.clear();          // Remove all
```

For DSA

Prefer:

```
Deque<Integer> stack = new ArrayDeque<>();
```

```
stack.push(x);
```

```
stack.peek();
```

```
stack.pop();
```

instead of:

```
Stack<Integer> stack = new Stack<>();
```

Iterator

```
Iterator<Integer> iterator = list.iterator();
```

```
iterator.hasNext(); // Check if next element exists
```

```
iterator.next(); // Get next element
```

```
iterator.remove(); // Remove last element returned by next()
```

```
iterator.forEachRemaining(x ->System.out.println(x) ); // Perform action on remaining elements
```

COLLECTIONS UTILITY CLASS

Collections is not a collection itself. It provides useful algorithms for collections.

```
Collections.sort(list);          // Sort ascending
```

```
Collections.reverse(list);       // Reverse list
```

```
Collections.shuffle(list);       // Randomly shuffle
```

```
Collections.max(list);           // Maximum  
Collections.min(list);           // Minimum  
Collections.frequency(list, x);   // Count occurrences  
Collections.swap(list, i, j);     // Swap two elements  
Collections.fill(list, x);        // Replace all elements  
Collections.binarySearch(list, x); // Binary search
```

For example:

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(5, 2, 8, 1));  
Collections.sort(list);  
System.out.println(list);  
  
// [1, 2, 5, 8]
```

Arrays UTILITY CLASS

```
Arrays.sort(arr);                // Sort array  
Arrays.binarySearch(arr, x);      // Binary search  
Arrays.toString(arr);            // Print array  
Arrays.fill(arr, x);             // Fill array  
Arrays.equals(arr1, arr2);        // Compare arrays  
Arrays.copyOf(arr, n);            // Copy array  
Arrays.copyOfRange(arr, l, r);    // Copy range
```

COMMON COLLECTION METHODS

```
list.addAll(otherList); // Add all elements  
list.removeAll(otherList); // Remove matching elements  
list.retainAll(otherList); // Keep only matching elements  
list.containsAll(otherList); // Check if all elements exist
```

MAP ITERATION

Iterating through keys

```
for (Integer key : map.keySet()) {  
    System.out.println(key);  
}
```

Iterating through values

```
for (String value : map.values()) {  
    System.out.println(value);  
}
```

Iterating through key-value pairs

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {  
    System.out.println(entry.getKey() + " " + entry.getValue());  
}
```

USEFUL MAP METHODS

```
map.replace(key, value); // Replace value
```

```
map.replaceAll((k, v) -> v + 1); // Update all values
```

```
map.computeIfAbsent(key, k -> 0); // Create value if key absent
```

```
map.merge(key, 1, Integer::sum); // Add/update value
```

Frequency Map

```
map.merge(x, 1, Integer::sum);
```

or

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

COMPARATOR

```
Collections.sort(list, Comparator.reverseOrder()); // Sort descending
```

```
list.sort((a, b) -> a - b); // Ascending
```

```
list.sort((a, b) -> b - a); // Descending
```

PriorityQueue with Comparator

```
PriorityQueue<Integer> pq =  
    new PriorityQueue<>(Comparator.reverseOrder());
```

Important: Comparator is used for custom sorting/order.

COMPARABLE vs COMPARATOR

Comparable → Defines natural/default sorting

Comparator → Defines custom sorting

WHICH COLLECTION TO USE?

ARRAYLIST

- Fast random access
- Use when you need list.get(i)

LINKEDLIST

- Add/remove from ends
- Can also work as Queue/Deque

HASHSET

- Unique elements
- Fast search
- No guaranteed order

LINKEDHASHSET

- Unique elements
- Maintains insertion order

TREESET

- Unique elements
- Automatically sorted
- Useful for lower(), higher(), floor(), ceiling()

HASHMAP

- Key → Value
- Fast lookup

LINKEDHASHMAP

- Key → Value
- Maintains insertion order

TREEMAP

- Key → Value
- Keys automatically sorted

QUEUE

- FIFO
- First In → First Out

ARRAYDEQUE

- Double-ended queue
- Can be used as Stack
- Preferred over Stack for DSA

PRIORITYQUEUE

→ Heap

→ Min Heap by default

→ Max Heap using `Comparator.reverseOrder()`

DSA QUICK REMINDER

`ArrayList` → Dynamic Array

`LinkedList` → Linked List / Queue / Deque

`HashSet` → Unique + Fast Search

`LinkedHashSet` → Unique + Insertion Order

`TreeSet` → Unique + Sorted

`HashMap` → Key → Value

`LinkedHashMap` → Key → Value + Insertion Order

`TreeMap` → Key → Value + Sorted Keys

`Queue` → FIFO

`ArrayDeque` → Stack / Deque

`PriorityQueue` → Heap

Most Important for DSA

`HashMap`:

`map.put(key, value);`

`map.get(key);`

`map.containsKey(key);`

`map.containsValue(value);`

`map.entrySet();`

`HashSet`:

`set.add(x);`

`set.contains(x);`

`set.remove(x);`

`Queue`:

`queue.offer(x);`

`queue.peek();`

`queue.poll();`

Stack using `ArrayDeque`:

`stack.push(x);`

`stack.peek();`

`stack.pop();`

`PriorityQueue`:

`pq.offer(x);`

`pq.peek();`

`pq.poll();`

`TreeSet`:

`set.lower(x);`

`set.higher(x);`

```
set.floor(x);  
set.ceiling(x);
```